Contents

1 Miscellaneous

1.1 What to Look for

```
Wrong answer:
Print your solution! (Print debug output, as well.)
Are you clearing all DS between test cases?
Can your algorithm handle the whole range of input?
Read the full problem statement again.
Do you handle all corner cases correctly?
Have you understood the problem correctly?
Any uninitialized variables? Any overflows?
Confusing N and M, i and j, etc.?
Are you sure your algorithm works?
What special cases have you not thought of?
Are you sure the STL functions you use work as you think?
Add some assertions, maybe resubmit.
Create some testcases to run your algorithm on.
Go through the algorithm for a simple case.
Go through this list again.
Go for a small walk, e.g. to the toilet. Is your output format correct? (including ↩
    whitespace)
Recode

Runtime error: In some judges, MLE is considered RTE (Check your memory use)
Have you tested all corner cases locally?
Any uninitialized variables?
Are you reading or writing outside the range of any vector?
Any assertions that might fail?
Infinite recursion?
Any possible division by 0? Ex. mod 0
Any possible infinite recursion?
Invalidated pointers or iterators?
Are you using too much memory?
Debug with resubmits.
Force WA (asserts for RTE problems)

Time limit exceeded:
Do you have any possible infinite loops?
What is the complexity of your algorithm?
Are you copying a lot of unnecessary data? (References)
How big is the input and output? (consider scanf)
Avoid vector, map. (use arrays/unordered_map)
What do your teammates think about your algorithm?

Memory limit exceeded:
What is the max amount of memory your algorithm should need?
Are you clearing all DS between test cases?
Infinite recursion? (As in pushing into a DS infinitely)
```

1.2 Day of Date

```
// 0-based
const vector<int> T = {0, 3, 2, 5, 0, 3, 5, 1, 4, 6, 2, 4}
int day(int d, int m, int y) {
  y -= (m < 3);
  return (y + y / 4 - y / 100 + y / 400 + T[m - 1] + d) % 7;
}
```

1.3 Number of Days since 1-1-1

```
int rdn(int d, int m, int y) {
  if(m < 3)
    --y, m += 12;
  return 365 * y + y / 4 - y / 100 + y / 400
         + (153 * m - 457) / 5 + d - 306;
}
```

1.4 Enumerate Subsets of a Bitmask

```
int x = 0;
do {
  // do stuff with the bitmask here
  x = (x + 1 + ~m) & m;
} while(x != 0);
```

1.5 Josephus Problem

```
ll josephus(ll n, ll k) { // O(k log n)
  if(n == 1) return 0;
  if(k == 1) return n - 1;
  if(k > n) return (josephus(n - 1, k) + k) % n;
  ll cnt = n / k;
  ll res = josephus(n - cnt, k);
  res -= n % k;
  if(res < 0) res += n;
  else res += res / (k - 1);
  return res;
}
int josephus(int n, int k) { // O(n)
  int res = 0;
  for(int i = 1; i <= n; ++i) res = (res + k) % i;
  return res + 1;
}
```

1.6 Random Primes

```
36671 74101 724729 825827 924997 1500005681 2010408371 2010405347
```

1.7 RNG

```
// RNG - rand_int(min, max), inclusive

mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());

template<class T>
T rand_int(T mn, T mx) {
  return uniform_int_distribution<T>(mn, mx)(rng);
}
```

2 Data Structures

2.1 Segment Tree

```
struct segment_tree {
    ll SZ=1;
    vector<ll> arr;
    ll default_value = 0;
    segment_tree(int _sz=(int)2.5e5) {
        while (SZ<_sz) {SZ<<=1;}
        arr.resize(SZ<<1);
        for (auto& x:arr) x=default_value;
    }

    ll merge(ll a, ll b) {
        return a+b;
    }

    void update(ll idx, ll val) {
        ll nd=idx+SZ;
        arr[nd]=val;
        while (nd>1) { //update par
            nd/=2;
            arr[nd] = merge(arr[2*nd], arr[2*nd+1]);
        }
    }

    ll query(ll nd, ll cl, ll cr, ll ql, ll qr) {
        if (ql<=cl && cr<=qr) return arr[nd];
        else if (cl>qr || cr<ql) return default_value;
        else {
            ll mid = (cl+cr)/2;
            return merge(query(2*nd, cl, mid, ql, qr), query(2*nd+1, mid+1, cr, ql, qr↩
                ));
        }
    }
};
```

2.2 STL PBDS

```
// ost = ordered set
// omp = ordered map
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace __gnu_pbds;

template<class T>
using ost = tree<T, null_type, less<T>, rb_tree_tag,
      tree_order_statistics_node_update>;
template<class T, class U>
using omp = tree<T, U, less<T>, rb_tree_tag,
      tree_order_statistics_node_update>;
```

2.3 Unordered Map Custom Hash

```
struct custom_hash {
  static uint64_t splitmix64(uint64_t x) {
    x += 0x9e3779b97f4a7c15;
    x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
    x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
    return x ^ (x >> 31);
  }
  size_t operator()(uint64_t x) const {
    static const uint64_t FIXED_RANDOM =
      chrono::steady_clock::now().time_since_epoch().count();
    return splitmix64(x + FIXED_RANDOM);
  }
};

unordered_map<int, int, custom_hash> umap;
```

2.4 Mo's on Tree

$ST(u) \leq ST(v)$
$P = LCA(u, v)$
If $P = u$, query $[ST(u), ST(v)]$
Else query $[EN(u), ST(v)] + [ST(P), ST(P)]$

3 Dynamic Programming

3.1 DP CHT

```
#include<complex> // Required!!!
typedef int ftype; // ll/ld/int will do
typedef complex<ftype> point;
#define x real
#define y imag

ftype dot(point a, point b) { return (conj(a) * b).x(); }
ftype cross(point a, point b) { return (conj(a) * b).y(); }

struct CHT {
    // Currently min, to switch to max, flip all <0 and >0.
    vector<point> hull, vecs;

    void add_line(ftype k, ftype b) {
        point nw = {k, b};
        while (!vecs.empty() && dot(vecs.back(), nw - hull.back()) < 0) {
            hull.pop_back();
```

```
            vecs.pop_back();
        }
        if (!hull.empty()) vecs.push_back(point(0, 1) * (nw - hull.back()));
        hull.push_back(nw);
    }

    ftype get(ftype x) const {
        point query = {x, 1};
        auto it = lower_bound(vecs.begin(), vecs.end(), query,
                              [](point a, point b) { return cross(a, b) > 0; });
        int idx = it - vecs.begin();
        if (idx == (int)hull.size()) idx--;
        return dot(query, hull[idx]);
    }
};
```

3.2 DP DNC

```
void f(int rem, int l, int r, int optl, int optr) {
  if(l > r)
    return;
  int mid = l + r >> 1;
  int opt = MOD, optid = mid;
  for(int i = optl; i <= mid && i <= optr; ++i) {
    if(dp[rem - 1][i] + c[i][mid] < opt) {
      opt = dp[rem - 1][i] + c[i][mid];
      optid = i;
    }
  }
  dp[rem][mid] = opt;
  f(rem, l, mid - 1, optl, optid);
  f(rem, mid + 1, r, optid, optr);
  return;
}
rep(i, 1, n)dp[1][i] = c[0][i];
rep(i, 2, k)f(i, i, n, i, n);
```

3.3 DP Knuth-Yao

```
// opt[i+1][j] <= opt[i][j] <= opt[i][j+1]
// dp[i][j] = min{k} dp[i][k]+dp[k][j]+cost[i][j]
for(int k = 0; k <= n; k++) {
  for(int i = 0; i + k <= n; i++) {
    if(k < 2)
      dp[i][i + k] = 0, opt[i][i + k] = i;
    else {
      int sta = opt[i][i + k - 1];
      int end = opt[i + 1][i + k];
      for(int j = sta; j <= end; j++) {
        if(dp[i][j] + dp[j][i + k] + cost[i][i + k] < dp[i][i + k]) {
          dp[i][i + k] = dp[i][j] + dp[j][i + k] + cost[i][i + k];
          opt[i][i + k] = j;
        }
      }
    }
  }
}
```

3.4 DP SOS

```
vector<int> sos(1 << n);
vector<vector<int>> dp(1 << n, vector<int>(n + 1));

for (int x = 0; x < (1 << n); x++) {
    dp[x][0] = a[x];
    for (int i = 0; i < n; i++) {
        dp[x][i + 1] = dp[x][i];
        if (x & (1 << i)) { dp[x][i + 1] += dp[x ^ (1 << i)][i]; }
    }
    sos[x] = dp[x][n];
}

// Alternative memory optimized
sos = a;

for (int i = 0; i < n; i++) {
    for (int x = 0; x < (1 << n); x++) {
        if (x & (1 << i)) { sos[x] += sos[x ^ (1 << i)]; }
    }
}
```

4 Geometry

4.1 Geometry Template

```
/*
TABLE OF CONTENT
0. Basic Rule
    0.1. Everything is in double
    0.2. Every comparison use EPS
    0.3. Every degree in rad
1. General Double Operation
    1.1. const double EPS=1E-9
    1.2. const double PI=acos(-1.0)
    1.3. const double INFD=1E9
    1.3. between_d(double x,double l,double r)
        check whether x is between l and r inclusive with EPS
    1.4. same_d(double x,double y)
        check whether x=y with EPS
    1.5. dabs(double x)
        absolute value of x
2. Point
    2.1. struct point
        2.1.1. double x,y
            cartesian coordinate of the point
        2.1.2. point()
            default constructor
        2.1.3. point(double _x,double _y)
            constructor, set the point to (_x,_y)
        2.1.4. bool operator< (point other)
            regular pair<double,double> operator < with EPS
        2.1.5. bool operator== (point other)
            regular pair<double,double> operator == with EPS
    2.2. hypot(point P)
        length of hypotenuse of point P to (0,0)
    2.3. e_dist(point P1,point P2)
        euclidean distance from P1 to P2
    2.4. m_dist(point P1,point P2)
        manhattan distance from P1 to P2
    2.5. point rotate(point P,point O,double angle)
        rotate point P from the origin O by angle ccw
3. Vector
    3.1. struct vec
        3.1.1. double x,y
            x and y magnitude of the vector
        3.1.2. vec()
            default constructor
        3.1.3. vec(double _x,double _y)
            constructor, set the vector to (_x,_y)
        3.1.4. vec(point A,point B)
            constructor, set the vector to vector AB (A->B)
*/
/*General Double Operation*/

const double PI = acos(-1.0);
const double INFD = 1E9;
double between_d(double x, double l, double r) {
```

```
  return (min(l, r) <= x + EPS && x <= max(l, r) + EPS);
}
double same_d(double x, double y) {
  return between_d(x, y, y);
}
double dabs(double x) {
  if(x < EPS)
    return -x;
  return x;
}
/*Point*/
struct point {
  double x, y;
  point() {
    x = y = 0.0;
  }
  point(double _x, double _y) {
    x = _x;
    y = _y;
  }
  bool operator< (point other) {
    if(x < other.x + EPS)
      return true;
    if(x + EPS > other.x)
      return false;
    return y < other.y + EPS;
  }
  bool operator== (point other) {
    return same_d(x, other.x) && same_d(y, other.y);
  }
};
double e_dist(point P1, point P2) {
  return hypot(P1.x - P2.x, P1.y - P2.y);
}
double m_dist(point P1, point P2) {
  return dabs(P1.x - P2.x) + dabs(P1.y - P2.y);
}
double pointBetween(point P, point L, point R) {
  return (e_dist(L, P) + e_dist(P, R) == e_dist(L, R));
}
bool collinear(point P, point L,
               point R) { //newly added(luis), cek 3 poin segaris
  return P.x * (L.y - R.y) + L.x * (R.y - P.y) + R.x * (P.y - L.y) ==
         0; // bole gnti “dabs(x)<”EPS
}

/*Vector*/
struct vec {
  double x, y;
  vec() {
    x = y = 0.0;
  }
  vec(double _x, double _y) {
    x = _x;
    y = _y;
  }
  vec(point A) {
    x = A.x;
    y = A.y;
  }
  vec(point A, point B) {
    x = B.x - A.x;
    y = B.y - A.y;
  }
};
vec scale(vec v, double s) {
  return vec(v.x * s, v.y * s);
}
vec flip(vec v) {
  return vec(-v.x, -v.y);
}
```



```
double dot(vec u, vec v) {
  return (u.x * v.x + u.y * v.y);
}
double cross(vec u, vec v) {
  return (u.x * v.y - u.y * v.x);
}
double norm_sq(vec v) {
  return (v.x * v.x + v.y * v.y);
}
point translate(point P, vec v) {
  return point(P.x + v.x, P.y + v.y);
}
point rotate(point P, point O, double angle) {
  vec v(O);
  P = translate(P, flip(v));
  return translate(point(P.x * cos(angle) - P.y * sin(angle),
                         P.x * sin(angle) + P.y * cos(angle)), v);
}
point mid(point P, point Q) {
  return point((P.x + Q.x) / 2, (P.y + Q.y) / 2);
}
double angle(point A, point O, point B) {
  vec OA(O, A), OB(O, B);
  return acos(dot(OA, OB) / sqrt(norm_sq(OA) * norm_sq(OB)));
}
int orientation(point P, point Q, point R) {
  vec PQ(P, Q), PR(P, R);
  double c = cross(PQ, PR);
  if(c < -EPS)
    return -1;
  if(c > EPS)
    return 1;
  return 0;
}
/*Line*/
struct line {
  double a, b, c;
  line() {
    a = b = c = 0.0;
  }
  line(double _a, double _b, double _c) {
    a = _a;
    b = _b;
    c = _c;
  }
  line(point P1, point P2) {
    if(P1 < P2)
      swap(P1, P2);
    if(same_d(P1.x, P2.x))
      a = 1.0, b = 0.0, c = -P1.x;
    else
      a = -(P1.y - P2.y) / (P1.x - P2.x), b = 1.0, c = -(a * P1.x) - P1.y;
  }
  line(point P, double slope) {
    if(same_d(slope, INFD))
      a = 1.0, b = 0.0, c = -P.x;
    else
      a = -slope, b = 1.0, c = -(a * P.x) - P.y;
  }
  bool operator== (line other) {
    return same_d(a, other.a) && same_d(b, other.b) && same_d(c, other.c);
  }
  double slope() {
    if(same_d(b, 0.0))
      return INFD;
    return -(a / b);
  }
};
bool paralel(line L1, line L2) {
  return same_d(L1.a, L2.a) && same_d(L1.b, L2.b);
}
```

```
bool intersection(line L1, line L2, point& P) {
  if(paralel(L1, L2))
    return false;
  P.x = (L2.b * L1.c - L1.b * L2.c) / (L2.a * L1.b - L1.a * L2.b);
  if(same_d(L1.b, 0.0))
    P.y = -(L2.a * P.x + L2.c);
  else
    P.y = -(L1.a * P.x + L1.c);
  return true;
}
double pointToLine(point P, point A, point B, point& C) {
  vec AP(A, P), AB(A, B);
  double u = dot(AP, AB) / norm_sq(AB);
  C = translate(A, scale(AB, u));
  return e_dist(P, C);
}
double lineToLine(line L1, line L2) {
  if(!paralel(L1, L2))
    return 0.0;
  return dabs(L2.c - L1.c) / sqrt(L1.a * L1.a + L1.b * L1.b);
}
/*Line Segment*/
struct segment {
  point P, Q;
  line L;
  segment() {
    point T1;
    P = Q = T1;
    line T2;
    L = T2;
  }
  segment(point _P, point _Q) {
    P = _P;
    Q = _Q;
    if(Q < P)
      swap(P, Q);
    line T(P, Q);
    L = T;
  }
  bool operator== (segment other) {
    return P == other.P && Q == other.Q;
  }
};
bool onSegment(point P, segment S) {
  if(orientation(S.P, S.Q, P) != 0)
    return false;
  return between_d(P.x, S.P.x, S.Q.x) && between_d(P.y, S.P.y, S.Q.y);
}
bool s_intersection(segment S1, segment S2) {
  double o1 = orientation(S1.P, S1.Q, S2.P);
  double o2 = orientation(S1.P, S1.Q, S2.Q);
  double o3 = orientation(S2.P, S2.Q, S1.P);
  double o4 = orientation(S2.P, S2.Q, S1.Q);
  if(o1 != o2 && o3 != o4)
    return true;
  if(o1 == 0 && onSegment(S2.P, S1))
    return true;
  if(o2 == 0 && onSegment(S2.Q, S1))
    return true;
  if(o3 == 0 && onSegment(S1.P, S2))
    return true;
  if(o4 == 0 && onSegment(S1.Q, S2))
    return true;
  return false;
}
double pointToSegment(point P, point A, point B, point& C) {
  vec AP(A, P), AB(A, B);
  double u = dot(AP, AB) / norm_sq(AB);
  if(u < EPS) {
    C = A;
    return e_dist(P, A);
  }
  if(u + EPS > 1.0) {
    C = B;
    return e_dist(P, B);
  }
  return pointToLine(P, A, B, C);
}
double segmentToSegment(segment S1, segment S2) {
  if(s_intersection(S1, S2))
    return 0.0;
  double ret = INFD;
  point dummy;
  ret = min(ret, pointToSegment(S1.P, S2.P, S2.Q, dummy));
  ret = min(ret, pointToSegment(S1.Q, S2.P, S2.Q, dummy));
  ret = min(ret, pointToSegment(S2.P, S1.P, S1.Q, dummy));
  ret = min(ret, pointToSegment(S2.Q, S1.P, S1.Q, dummy));
  return ret;
}
/*Circle*/
struct circle {
  point P;
  double r;
  circle() {
    point P1;
    P = P1;
    r = 0.0;
  }
  circle(point _P, double _r) {
    P = _P;
    r = _r;
  }
  circle(point P1, point P2) {
    P = mid(P1, P2);
    r = e_dist(P, P1);
  }
  circle(point P1, point P2, point P3) {
    vector<point> T;
    T.clear();
    T.pb(P1);
    T.pb(P2);
    T.pb(P3);
    sort(T.begin(), T.end());
    P1 = T[0];
    P2 = T[1];
    P3 = T[2];
    point M1, M2;
    M1 = mid(P1, P2);
    M2 = mid(P2, P3);
    point Q2, Q3;
    Q2 = rotate(P2, P1, PI / 2);
    Q3 = rotate(P3, P2, PI / 2);
    vec P1Q2(P1, Q2), P2Q3(P2, Q3);
    point M3, M4;
    M3 = translate(M1, P1Q2);
    M4 = translate(M2, P2Q3);
    line L1(M1, M3), L2(M2, M4);
    intersection(L1, L2, P);
    r = e_dist(P, P1);
  }
  bool operator==(circle other) {
    return (P == other.P && same_d(r, other.r));
  }
};
bool insideCircle(point P, circle C) {
  return e_dist(P, C.P) <= C.r + EPS;
}
bool c_intersection(circle C1, circle C2, point& P1, point& P2) {
  double d = e_dist(C1.P, C2.P);
  if(d > C1.r + C2.r) {
    return false;  //d+EPS kalo butuh
  }
```

```
  if(d < dabs(C1.r - C2.r) + EPS)
    return false;
  double x1 = C1.P.x, y1 = C1.P.y, r1 = C1.r, x2 = C2.P.x, y2 = C2.P.y, r2 = C2.r;
  double a = (r1 * r1 - r2 * r2 + d * d) / (2 * d), h = sqrt(r1 * r1 - a * a);
  point T(x1 + a * (x2 - x1) / d, y1 + a * (y2 - y1) / d);
  P1 = point(T.x - h * (y2 - y1) / d, T.y + h * (x2 - x1) / d);
  P2 = point(T.x + h * (y2 - y1) / d, T.y - h * (x2 - x1) / d);
  return true;
}
bool lc_intersection(line L, circle O, point& P1, point& P2) {
  double a = L.a, b = L.b, c = L.c, x = O.P.x, y = O.P.y, r = O.r;
  double A = a * a + b * b, B = 2 * a * b * y - 2 * a * c - 2 * b * b * x,
         C = b * b * x * x + b * b * y * y - 2 * b * c * y + c * c - b * b * r * r;
  double D = B * B - 4 * A * C;
  point T1, T2;
  if(same_d(b, 0.0)) {
    T1.x = c / a;
    if(dabs(x - T1.x) + EPS > r)
      return false;
    if(same_d(T1.x - r - x, 0.0) || same_d(T1.x + r - x, 0.0)) {
      P1 = P2 = point(T1.x, y);
      return true;
    }
    double dx = dabs(T1.x - x), dy = sqrt(r * r - dx * dx);
    P1 = point(T1.x, y - dy);
    P2 = point(T1.x, y + dy);
    return true;
  }
  if(same_d(D, 0.0)) {
    T1.x = -B / (2 * A);
    T1.y = (c - a * T1.x) / b;
    P1 = P2 = T1;
    return true;
  }
  if(D < EPS)
    return false;
  D = sqrt(D);
  T1.x = (-B - D) / (2 * A);
  T1.y = (c - a * T1.x) / b;
  P1 = T1;
  T2.x = (-B + D) / (2 * A);
  T2.y = (c - a * T2.x) / b;
  P2 = T2;
  return true;
}
bool sc_intersection(segment S, circle C, point& P1, point& P2) {
  bool cek = lc_intersection(S.L, C, P1, P2);
  if(!cek)
    return false;
  double x1 = S.P.x, y1 = S.P.y, x2 = S.Q.x, y2 = S.Q.y;
  bool b1 = between_d(P1.x, x1, x2) && between_d(P1.y, y1, y2);
  bool b2 = between_d(P2.x, x1, x2) && between_d(P2.y, y1, y2);
  if(P1 == P2)
    return b1;
  if(b1 || b2) {
    if(!b1)
      P1 = P2;
    if(!b2)
      P2 = P1;
    return true;
  }
  return false;
}
/*Triangle*/
double t_perimeter(point A, point B, point C) {
  return e_dist(A, B) + e_dist(B, C) + e_dist(C, A);
}
double t_area(point A, point B, point C) {
  double s = t_perimeter(A, B, C) / 2;
  double ab = e_dist(A, B), bc = e_dist(B, C), ac = e_dist(C, A);
  return sqrt(s * (s - ab) * (s - bc) * (s - ac));
}
circle t_inCircle(point A, point B, point C) {
  vector<point> T;
  T.clear();
  T.pb(A);
  T.pb(B);
  T.pb(C);
  sort(T.begin(), T.end());
  A = T[0];
  B = T[1];
  C = T[2];
  double r = t_area(A, B, C) / (t_perimeter(A, B, C) / 2);
  double ratio = e_dist(A, B) / e_dist(A, C);
  vec BC(B, C);
  BC = scale(BC, ratio / (1 + ratio));
  point P;
  P = translate(B, BC);
  line AP1(A, P);
  ratio = e_dist(B, A) / e_dist(B, C);
  vec AC(A, C);
  AC = scale(AC, ratio / (1 + ratio));
  P = translate(A, AC);
  line BP2(B, P);
  intersection(AP1, BP2, P);
  return circle(P, r);
}
circle t_outCircle(point A, point B, point C) {
  return circle(A, B, C);
}
/*Polygon*/
struct polygon {
  vector<point> P;
  polygon() {
    P.clear();
  }
  polygon(vector<point>& _P) {
    P = _P;
  }
};
bool rayCast(point P, polygon& A) {
  point Q(P.x, 10000);
  line cast(P, Q);
  int cnt = 0;
  FOR(i, (int)(A.P.size()) - 1) {
    line temp(A.P[i], A.P[i + 1]);
    point I;
    bool B = intersection(cast, temp, I);
    if(!B)
      continue;
    else if(I == A.P[i] || I == A.P[i + 1])
      continue;
    else if(pointBetween(I, A.P[i], A.P[i + 1]) && pointBetween(I, P, Q))
      cnt++;
  }
  return cnt % 2 == 1;
}
// line segment p-q intersect with line A-B.
point lineIntersectSeg(point p, point q, point A, point B) {
  double a = B.y - A.y;
  double b = A.x - B.x;
  double c = B.x * A.y - A.x * B.y;
  double u = fabs(a * p.x + b * p.y + c);
  double v = fabs(a * q.x + b * q.y + c);
  return point((p.x * v + q.x * u) / (u + v), (p.y * v + q.y * u) / (u + v));
}
// cuts polygon Q along the line formed by point a -> point b
// (note: the last point must be the same as the first point)
vector<point> cutPolygon(point a, point b, const vector<point>& Q) {
  vector<point> P;
  for(int i = 0; i < (int)Q.size(); i++) {
    double left1 = cross(toVec(a, b), toVec(a, Q[i]));
```

```
    double left2 = 0;
    if(i != (int)Q.size() - 1)
      left2 = cross(toVec(a, b), toVec(a, Q[i + 1]));
    if(left1 > -EPS)
      P.push_back(Q[i]);
    if(left1 * left2 < -EPS)
      P.push_back(lineIntersectSeg(Q[i], Q[i + 1], a, b));
  }
  if(!P.empty() && !(P.back() == P.front()))
    P.push_back(P.front());
  return P;
}
circle minCoverCircle(polygon& A) {
  vector<point> p = A.P;
  point c;
  circle ret;
  double cr = 0.0;
  int i, j, k;
  c = p[0];
  for(i = 1; i < p.size(); i++) {
    if(e_dist(p[i], c) >= cr + EPS) {
      c = p[i], cr = 0;
      ret = circle(c, cr);
      for(j = 0; j < i; j++) {
        if(e_dist(p[j], c) >= cr + EPS) {
          c = mid(p[i], p[j]);
          cr = e_dist(p[i], c);
          ret = circle(c, cr);
          for(k = 0; k < j; k++) {
            if(e_dist(p[k], c) >= cr + EPS) {
              ret = circle(p[i], p[j], p[k]);
              c = ret.P;
              cr = ret.r;
            }
          }
        }
      }
    }
  }
  return ret;
}
/*Geometry Algorithm*/
double DP[110][110];
double minCostPolygonTriangulation(polygon& A) {
  if(A.P.size() < 3)
    return 0;
  FOR(i, A.P.size()) {
    for(int j = 0, k = i; k < A.P.size(); j++, k++) {
      if(k < j + 2)
        DP[j][k] = 0.0;
      else {
        DP[j][k] = INFD;
        REP(l, j + 1, k - 1) {
          double cost = e_dist(A.P[j], A.P[k]) + e_dist(A.P[k], A.P[l]) + e_dist(A.P[l↩
              ],
                        A.P[j]);
          DP[j][k] = min(DP[j][k], DP[j][l] + DP[l][k] + cost);
        }
      }
    }
  }
  return DP[0][A.P.size() - 1];
}
```

4.2 Convex Hull

```
typedef double TD;                  // for precision shits
namespace GEOM {
  typedef pair<TD, TD> Pt;          // vector and points
  const TD EPS = 1e-9;
  const TD maxD = 1e9;
  TD cross(Pt a, Pt b, Pt c) {      // right hand rule
    TD v1 = a.first - c.first;      // (a-c) X (b-c)
    TD v2 = a.second - c.second;
    TD u1 = b.first - c.first;
    TD u2 = b.second - c.second;
    return v1 * u2 - v2 * u1;
  }
  TD cross(Pt a, Pt b) {            // a X b
    return a.first * b.second - a.second * b.first;
  }
  TD dot(Pt a, Pt b, Pt c) {        // (a-c) . (b-c)
    TD v1 = a.first - c.first;
    TD v2 = a.second - c.second;
    TD u1 = b.first - c.first;
    TD u2 = b.second - c.second;
    return v1 * u1 + v2 * u2;
  }
  TD dot(Pt a, Pt b) {              // a . b
    return a.first * b.first + a.second * b.second;
  }
  TD dist(Pt a, Pt b) {
    return sqrt((a.first - b.first) * (a.first - b.first) +
                (a.second - b.second) * (a.second - b.second));
  }
  TD shoelaceX2(vector<Pt>& convHull) {
    TD ret = 0;
    for(int i = 0, n = convHull.size(); i < n; i++)
      ret += cross(convHull[i], convHull[(i + 1) % n]);
    return ret;
  }
  vector<Pt> createConvexHull(vector<Pt>& points) {
    sort(points.begin(), points.end());
    vector<Pt> ret;
    for(int i = 0; i < points.size(); i++) {
      while(ret.size() > 1 &&
            cross(points[i], ret[ret.size() - 1], ret[ret.size() - 2]) < -EPS)
        ret.pop_back();
      ret.push_back(points[i]);
    }
    for(int i = points.size() - 2, sz = ret.size(); i >= 0; i--) {
      while(ret.size() > sz &&
            cross(points[i], ret[ret.size() - 1], ret[ret.size() - 2]) < -EPS)
        ret.pop_back();
      if(i == 0)
        break;
      ret.push_back(points[i]);
    }
    return ret;
    bool isInside(Pt pv, vector<Pt>& x) { //using winding number
      int n = x.size(), wn = 0;
      x.push_back(x[0]);
      for(int i = 0; i < n; ++i) {
        if(((x[i + 1].first <= pv.first && x[i].first >= pv.first) ||
            (x[i + 1].first >= pv.first && x[i].first <= pv.first)) &&
            ((x[i + 1].second <= pv.second && x[i].second >= pv.second) ||
             (x[i + 1].second >= pv.second && x[i].second <= pv.second))) {
          if(cross(x[i], x[i + 1], pv) == 0) {
            x.pop_back();
            return true;
          }
        }
      }
      for(int i = 0; i < n; ++i) {
        if(x[i].second <= pv.second) {
          if(x[i + 1].second > pv.second && cross(x[i], x[i + 1], pv) > 0)
            ++wn;
        } else if(x[i + 1].second <= pv.second && cross(x[i], x[i + 1], pv) < 0)
          --wn;
      }
      x.pop_back();
```

```
      return wn != 0;
    }
  }
  bool isInside(Pt pv, vector<Pt>& x) { //using winding number
    int n = x.size(), wn = 0;
    x.push_back(x[0]);
    for(int i = 0; i < n; ++i) {
      if(((x[i + 1].first <= pv.first && x[i].first >= pv.first) ||
          (x[i + 1].first >= pv.first && x[i].first <= pv.first)) &&
          ((x[i + 1].second <= pv.second && x[i].second >= pv.second) ||
           (x[i + 1].second >= pv.second && x[i].second <= pv.second))) {
        if(cross(x[i], x[i + 1], pv) == 0) {
          x.pop_back();
          return true;
        }
      }
    }
    for(int i = 0; i < n; ++i) {
      if(x[i].second <= pv.second) {
        if(x[i + 1].second > pv.second && cross(x[i], x[i + 1], pv) > 0)
          ++wn;
      } else if(x[i + 1].second <= pv.second && cross(x[i], x[i + 1], pv) < 0)
        --wn;
    }
    x.pop_back();
    return wn != 0;
  }
}
```

4.3 Closest Pair of Points

```
#define fi first
#define se second
typedef pair<int, int> pii;
struct Point {
  int x, y, id;
};
int compareX(const void* a, const void* b) {
  Point* p1 = (Point*)a, * p2 = (Point*)b;
  return (p1->x - p2->x);
}
int compareY(const void* a, const void* b) {
  Point* p1 = (Point*)a, * p2 = (Point*)b;
  return (p1->y - p2->y);
}
double dist(Point p1, Point p2) {
  return sqrt((double)(p1.x - p2.x) * (p1.x - p2.x) +
              (double)(p1.y - p2.y) * (p1.y - p2.y)
             );
}
pair<pii, double> bruteForce(Point P[], int n) {
  double min = 1e8;
  pii ret = pii(-1, -1);
  for(int i = 0; i < n; ++i)
    for(int j = i + 1; j < n; ++j)
      if(dist(P[i], P[j]) < min) {
        ret = pii(P[i].id, P[j].id);
        min = dist(P[i], P[j]);
      }
  return pair<pii, double> (ret, min);
}
pair<pii, double> getmin(pair<pii, double> x, pair<pii, double> y) {
  if(x.fi.fi == -1 && x.fi.se == -1)
    return y;
  if(y.fi.fi == -1 && y.fi.se == -1)
    return x;
  return (x.se < y.se) ? x : y;
}
pair<pii, double> stripClosest(Point strip[], int size, double d) {
  double min = d;
  pii ret = pii(-1, -1);
  qsort(strip, size, sizeof(Point), compareY);
  for(int i = 0; i < size; ++i)
    for(int j = i + 1; j < size && (strip[j].y - strip[i].y) < min; ++j)
      if(dist(strip[i], strip[j]) < min) {
        ret = pii(strip[i].id, strip[j].id);
        min = dist(strip[i], strip[j]);
      }
  return pair<pii, double>(ret, min);
}
pair<pii, double> closestUtil(Point P[], int n) {
  if(n <= 3)
    return bruteForce(P, n);
  int mid = n / 2;
  Point midPoint = P[mid];
  pair<pii, double> dl = closestUtil(P, mid);
  pair<pii, double> dr = closestUtil(P + mid, n - mid);
  pair<pii, double> d = getmin(dl, dr);
  Point strip[n];
  int j = 0;
  for(int i = 0; i < n; i++)
    if(abs(P[i].x - midPoint.x) < d.second)
      strip[j] = P[i], j++;
  return getmin(d, stripClosest(strip, j, d.second));
}
pair<pii, double> closest(Point P[], int n) {
  qsort(P, n, sizeof(Point), compareX);
  return closestUtil(P, n);
}
Point P[50005];
int main() {
  int n;
  scanf("%d", &n);
  for(int a = 0; a < n; a++) {
    scanf("%d%d", &P[a].x, &P[a].y);
    P[a].id = a;
  }
  pair<pii, double> hasil = closest(P, n);
  if(hasil.fi.fi > hasil.fi.se)
    swap(hasil.fi.fi, hasil.fi.se);
  printf("%d %d %.6lf\n", hasil.fi.fi, hasil.fi.se, hasil.se);
  return 0;
}
```

4.4 Smallest Enclosing Circle

```
// welzl's algo to find the 2d minimum enclosing circle of a set of points
// expected O(N)
// directions: remove duplicates and shuffle points, then call welzl(points)

struct Point {
  double x;
  double y;
};

struct Circle {
  double x, y, r;
  Circle() {}
  Circle(double _x, double _y, double _r): x(_x), y(_y), r(_r) {}
};

Circle trivial(const vector<Point>& r) {
  if(r.size() == 0)
    return Circle(0, 0, -1);
  else if(r.size() == 1)
    return Circle(r[0].x, r[0].y, 0);
  else if(r.size() == 2) {
    double cx = (r[0].x + r[1].x) / 2.0, cy = (r[0].y + r[1].y) / 2.0;
    double rad = hypot(r[0].x - r[1].x, r[0].y - r[1].y) / 2.0;
    return Circle(cx, cy, rad);
```

```
  } else {
    double x0 = r[0].x, x1 = r[1].x, x2 = r[2].x;
    double y0 = r[0].y, y1 = r[1].y, y2 = r[2].y;
    double d = (x0 - x2) * (y1 - y2) - (x1 - x2) * (y0 - y2);
    double cx = (((x0 - x2) * (x0 + x2) + (y0 - y2) * (y0 + y2)) / 2 *
                 (y1 - y2) - ((x1 - x2) * (x1 + x2) + (y1 - y2) * (y1 + y2)) / 2
                 * (y0 - y2)) / d;
    double cy = (((x1 - x2) * (x1 + x2) + (y1 - y2) * (y1 + y2)) / 2 *
                 (x0 - x2) - ((x0 - x2) * (x0 + x2) + (y0 - y2) * (y0 + y2)) / 2
                 * (x1 - x2)) / d;
    return Circle(cx, cy, hypot(x0 - cx, y0 - cy));
  }
}

// SHUFFLE THE POINTS FIRST!!!!!!
Circle welzl(const vector<Point>& p, int idx = 0, vector<Point> r = {}) {
  if(idx == (int) p.size() || r.size() == 3)
    return trivial(r);
  Circle d = welzl(p, idx + 1, r);
  if(hypot(p[idx].x - d.x, p[idx].y - d.y) > d.r) {
    r.push_back(p[idx]);
    d = welzl(p, idx + 1, r);
  }
  return d;
}
```

4.5 Centroid of Polygon

$C_x = \frac{1}{6A} \sum_{i=0}^{n-1} (x_i + x_{i+1})(x_i\ y_{i+1} - x_{i+1}\ y_i)$
$C_y = \frac{1}{6A} \sum_{i=0}^{n-1} (y_i + y_{i+1})(x_i\ y_{i+1} - x_{i+1}\ y_i)$

4.6 Pick Theorem

A: Area of a simply closed lattice polygon
B: Number of lattice points on the edges
I: Number of points in the interior
$A = I + \frac{B}{2} - 1$

5 Graphs

5.1 Articulation Point and Bridge

```
// gr -> adj list
// vector vis, low -> initialize to -1
// int timer -> initialize to 0
void dfs(int pos, int dad = -1) {
  vis[pos] = low[pos] = timer++;
  int kids = 0;
  for(auto& i : gr[pos]) {
    if(i == dad)
      continue;
    if(vis[i] >= 0)
      low[pos] = min(low[pos], vis[i]);
    else {
      dfs(i, pos);
      low[pos] = min(low[pos], low[i]);
      if(low[i] > vis[pos])
        is_bridge(pos, i)
      if(low[i] >= vis[pos] && dad >= 0)
        is_articulation_point(pos)
        ++kids;
    }
  }
  if(dad == -1 && kids > 1)
    is_articulation_point(pos)
  }
```

5.2 SCC and Strong Orientation

```
#define N 10020
vector<int> adj[N];
bool vis[N], ins[N];
int disc[N], low[N], gr[N];
stack<int> st;
int id, grid;
void scc(int cur, int par) {
  disc[cur] = low[cur] = ++id;
  vis[cur] = ins[cur] = 1;
  st.push(cur);
  for(int to : adj[cur]) {
    //if (to==par) continue; // ini untuk SO(scc undirected)
    if(!vis[to])
      scc(to, cur);
    if(ins[to])
      low[cur] = min(low[cur], low[to]);
  }
  if(low[cur] == disc[cur]) {
    grid++; // group id
    while(ins[cur]) {
      gr[st.tp] = grid;
      ins[st.tp] = 0;
      st.pop();
    }
  }
}
```

5.3 Dinic's Tourist

```
template <typename T>
class flow_graph {
 public:
  static constexpr T eps = (T) 1e-9;

  struct edge {
    int from;
    int to;
    T c;
    T f;
  };

  vector<vector<int>> g;
  vector<edge> edges;
  int n;
  int st;
  int fin;
  T flow;

  flow_graph(int _n, int _st, int _fin) : n(_n), st(_st), fin(_fin) {
    assert(0 <= st && st < n && 0 <= fin && fin < n && st != fin);
    g.resize(n);
    flow = 0;
  }

  void clear_flow() {
    for (const edge &e : edges) {
      e.f = 0;
    }
    flow = 0;
  }

  int add(int from, int to, T forward_cap, T backward_cap) {
    assert(0 <= from && from < n && 0 <= to && to < n);
    int id = (int) edges.size();
    g[from].push_back(id);
    edges.push_back({from, to, forward_cap, 0});
    g[to].push_back(id + 1);
    edges.push_back({to, from, backward_cap, 0});
```

```
    return id;
  }
};

template <typename T>
class dinic {
 public:
  flow_graph<T> &g;

  vector<int> ptr;
  vector<int> d;
  vector<int> q;

  dinic(flow_graph<T> &_g) : g(_g) {
    ptr.resize(g.n);
    d.resize(g.n);
    q.resize(g.n);
  }

  bool expath() {
    fill(d.begin(), d.end(), -1);
    q[0] = g.fin;
    d[g.fin] = 0;
    int beg = 0, end = 1;
    while (beg < end) {
      int i = q[beg++];
      for (int id : g.g[i]) {
        const auto &e = g.edges[id];
        const auto &back = g.edges[id ^ 1];
        if (back.c - back.f > g.eps && d[e.to] == -1) {
          d[e.to] = d[i] + 1;
          if (e.to == g.st) {
            return true;
          }
          q[end++] = e.to;
        }
      }
    }
    return false;
  }

  T dfs(int v, T w) {
    if (v == g.fin) {
      return w;
    }
    int &j = ptr[v];
    while (j >= 0) {
      int id = g.g[v][j];
      const auto &e = g.edges[id];
      if (e.c - e.f > g.eps && d[e.to] == d[v] - 1) {
        T t = dfs(e.to, min(e.c - e.f, w));
        if (t > g.eps) {
          g.edges[id].f += t;
          g.edges[id ^ 1].f -= t;
          return t;
        }
      }
      j--;
    }
    return 0;
  }

  T max_flow() {
    while (expath()) {
      for (int i = 0; i < g.n; i++) {
        ptr[i] = (int) g.g[i].size() - 1;
      }
      T big_add = 0;
      while (true) {
        T add = dfs(g.st, numeric_limits<T>::max());
        if (add <= g.eps) {
          break;
        }
        big_add += add;
      }
      if (big_add <= g.eps) {
        break;
      }
      g.flow += big_add;
    }
    return g.flow;
  }

  vector<bool> min_cut() {
    max_flow();
    vector<bool> ret(g.n);
    for (int i = 0; i < g.n; i++) {
      ret[i] = (d[i] != -1);
    }
    return ret;
  }
};
```

5.4 Bipartite Graphs

|MCBM| = Maximum Cardinality Bipartite Matching
|Minimum Vertex Cover| = |MCBM|
|Maximum Independent Set| = |V| - |MCBM|
|Minimum Path Cover| = |V| - |MCBM|
In a bipartite graph, a set of vertices S is a vertex cover if and only if its complement V-S is an independent set.
In a DAG, a set of vertices P is a path cover if and only if its complement V-P is an independent set.
To find the Minimum Vertex Cover from the MCBM, find all vertices not matched on the left side, perform a DFS traversing

5.5 Dinic's Maximum Flow

```
// O(VE log(max_flow)) if scaling == 1
// O((V + E) sqrt(E)) if unit graph (turn scaling off)
// O((V + E) sqrt(V)) if bipartite matching (turn scaling off)
// indices are 0-based
const ll INF = 1e18;

struct Dinic {
  struct Edge {
    int v;
    ll cap, flow;
    Edge(int _v, ll _cap): v(_v), cap(_cap), flow(0) {}
  };

  int n;
  ll lim;
  vector<vector<int>> gr;
  vector<Edge> e;
  vector<int> idx, lv;

  bool has_path(int s, int t) {
    queue<int> q;
    q.push(s);
    lv.assign(n, -1);
    lv[s] = 0;
    while(!q.empty()) {
      int c = q.front();
      q.pop();
      if(c == t)
        break;
      for(auto& i : gr[c]) {
        ll cur_flow = e[i].cap - e[i].flow;
        if(lv[e[i].v] == -1 && cur_flow >= lim) {
```

```
          lv[e[i].v] = lv[c] + 1;
          q.push(e[i].v);
        }
      }
    }
    return lv[t] != -1;
  }

  ll get_flow(int s, int t, ll left) {
    if(!left || s == t)
      return left;
    while(idx[s] < (int) gr[s].size()) {
      int i = gr[s][idx[s]];
      if(lv[e[i].v] == lv[s] + 1) {
        ll add = get_flow(e[i].v, t, min(left, e[i].cap - e[i].flow));
        if(add) {
          e[i].flow += add;
          e[i ^ 1].flow -= add;
          return add;
        }
      }
      ++idx[s];
    }
    return 0;
  }

  Dinic(int vertices, bool scaling = 1) : // toggle scaling here
    n(vertices), lim(scaling ? 1 << 30 : 1), gr(n) {}

  void add_edge(int from, int to, ll cap, bool directed = 1) {
    gr[from].push_back(e.size());
    e.emplace_back(to, cap);
    gr[to].push_back(e.size());
    e.emplace_back(from, directed ? 0 : cap);
  }

  ll get_max_flow(int s, int t) { // call this
    ll res = 0;
    while(lim) {  // scaling
      while(has_path(s, t)) {
        idx.assign(n, 0);
        while(ll add = get_flow(s, t, INF))
          res += add;
      }
      lim >>= 1;
    }
    return res;
  }
};
```

5.6 Minimum Cost Maximum Flow

```
using FlowT = ll;
using CostT = ll;

const FlowT F_INF = 1e18;
const CostT C_INF = 1e18;
const int MAX_V = 1e5 + 5;
const int MAX_E = 1e6 + 5;

namespace MCMF {
  int n, E;
  int adj[MAX_E], nxt[MAX_E], lst[MAX_V], frm[MAX_V], vis[MAX_V];
  FlowT cap[MAX_E], flw[MAX_E], totalFlow;
  CostT cst[MAX_E], dst[MAX_V], totalCost;

  void init(int _n) {
    n = _n;
    fill_n(lst, n, -1), E = 0;
  }
  void add(int u, int v, FlowT ca, CostT co) {
    adj[E] = v, cap[E] = ca, flw[E] = 0, cst[E] = +co;
    nxt[E] = lst[u], lst[u] = E++;
    adj[E] = u, cap[E] =  0, flw[E] = 0, cst[E] = -co;
    nxt[E] = lst[v], lst[v] = E++;
  }
  int spfa(int s, int t) {
    fill_n(dst, n, C_INF), dst[s] = 0;
    queue<int> que;
    que.push(s);
    while(que.size()) {
      int u = que.front();
      que.pop();
      for(int e = lst[u]; e != -1; e = nxt[e])
        if(flw[e] < cap[e]) {
          int v = adj[e];
          if(dst[v] > dst[u] + cst[e]) {
            dst[v] = dst[u] + cst[e];
            frm[v] = e;
            if(!vis[v]) {
              vis[v] = 1;
              que.push(v);
            }
          }
        }
      vis[u] = 0;
    }
    return dst[t] < C_INF;
  }
  pair<FlowT, CostT> solve(int s, int t) {
    totalCost = 0, totalFlow = 0;
    while(1) {
      if(!spfa(s, t))
        break;
      FlowT mn = F_INF;
      for(int v = t, e = frm[v]; v != s; v = adj[e ^ 1], e = frm[v])
        mn = min(mn, cap[e] - flw[e]);
      for(int v = t, e = frm[v]; v != s; v = adj[e ^ 1], e = frm[v]) {
        flw[e] += mn;
        flw[e ^ 1] -= mn;
      }
      totalFlow += mn;
      totalCost += mn * dst[t];
    }
    return {totalFlow, totalCost};
  }
};
```

5.7 Flows with Demands

```
let S0 be the source and T0 be the original sink
1. add 2 additional nodes, call them S1 and T1
2. connect S0 to nodes normally
3. connect nodes to T0 normally
4. for each edge(U, V), cap = original cap - demand
5. for each node N :
   1. add an edge(S1, N), cap = sum of inward demand to N
   2. add an edge(N, T1), cap = sum of outward demand from N
6. add an edge(T0, S0), cap = INF
7. the above is not a typo!
8. run max flow normally
9. for each edge(S1, V) and (U, T1), check if flow == cap
if step #9 fails, then it is not possible to satisfy the given demand
```

Mathematically, let $d(e)$ be the demand of edge e. Let V be the set of every vertex in the graph.

- $c'(S_1, v) = \sum_{u \in V} d(u, v)$ for each edge (s', v).
- $c'(v, T_1) = \sum_{v \in V} d(v, w)$ for each edge (v, t').
- $c'(u, v) = c(u, v) - d(u, v)$ for each edge (u, v) in the old network.

- $c'(T_0, S_0) = \infty$

5.8 Eulerian Path or Cycle

```
// finds a eulerian path / cycle
// visits each edge only once
// properties:
// - cycle: degrees are even
// - path: degrees are even OR degrees are even except for 2 vertices
// how to use: g = adjacency list g[n] = connected to n, undirected
// if there is a vertex u with an odd degree, call dfs(u)
// else call on any vertex
// ans = path result

vector<set<int>> g;
vector<int> ans;

void dfs(int u) {
  while(g[u].size()) {
    int v = *g[u].begin();
    g[u].erase(v);
    g[v].erase(u);
    dfs(v);
  }
  ans.push_back(u);
}
```

5.9 2-SAT

```
struct TwoSAT {
  int n;
  vector<vector<int>> g, gr;
  vector<int> comp, topological_order, answer;
  vector<bool> vis;

  TwoSAT() {}
  TwoSAT(int _n) :
    n(_n), g(2 * n), gr(2 * n), comp(2 * n), answer(2 * n), vis(2 * n) {}

  void add_edge(int u, int v) {
    g[u].push_back(v);
    gr[v].push_back(u);
  }

  // For the following three functions
  // int x, bool val: if 'val' is true, we take the variable to be x.
  // Otherwise we take it to be x's complement.

  // At least one of them is true
  void add_clause_or(int i, bool f, int j, bool p) {
    add_edge(i + (f ? n : 0), j + (p ? 0 : n));
    add_edge(j + (p ? n : 0), i + (f ? 0 : n));
  }

  // Only one of them is true
  void add_clause_xor(int i, bool f, int j, bool p) {
    add_clause_or(i, f, j, p);
    add_clause_or(i, !f, j, !p);
  }

  // Both of them have the same value
  void add_clause_and(int i, bool f, int j, bool p) {
    add_clause_xor(i, !f, j, p);
  }

  // Topological sort
  void dfs(int u) {
    vis[u] = true;
    for(const auto& v : g[u])
      if(!vis[v])
        dfs(v);
    topological_order.push_back(u);
  }

  // Extracting strongly connected components
  void scc(int u, int id) {
    vis[u] = true;
    comp[u] = id;
    for(const auto& v : gr[u])
      if(!vis[v])
        scc(v, id);
  }

  bool satisfiable() {
    fill(vis.begin(), vis.end(), false);
    for(int i = 0; i < 2 * n; i++)
      if(!vis[i])
        dfs(i);
    fill(vis.begin(), vis.end(), false);
    reverse(topological_order.begin(), topological_order.end());
    int id = 0;
    for(const auto& v : topological_order)
      if(!vis[v])
        scc(v, id++);
    // Constructing the answer
    for(int i = 0; i < n; i++) {
      if(comp[i] == comp[i + n])
        return false;
      answer[i] = (comp[i] > comp[i + n] ? 1 : 0);
    }
    return true;
  }
};
```

6 Math

6.1 Extended Euclidean GCD

```
// computes x and y such that ax + by = gcd(a, b) in O(log (min(a, b)))
// returns {gcd(a, b), x, y}
tuple<int, int, int> gcd(int a, int b) {
  if(b == 0) return {a, 1, 0};
  auto [d, x1, y1] = gcd(b, a % b);
  return {d, y1, x1 - y1* (a / b)};
}
```

6.2 Generalized CRT

```
template<typename T>
T extended_euclid(T a, T b, T& x, T& y) {
  if(b == 0) {
    x = 1;
    y = 0;
    return a;
  }
  T xx, yy, gcd;
  gcd = extended_euclid(b, a % b, xx, yy);
  x = yy;
  y = xx - (yy * (a / b));
  return gcd;
}
template<typename T>
T MOD(T a, T b) {
  return (a % b + b) % b;
}
// return x, lcm. x = a % n && x = b % m
```

```
template<typename T>
pair<T, T> CRT(T a, T n, T b, T m) {
  T _n, _m;
  T gcd = extended_euclid(n, m, _n, _m);
  if(n == m) {
    if(a == b)
      return pair<T, T>(a, n);
    else
      return pair<T, T>(-1, -1);
  } else if(abs(a - b) % gcd != 0)
    return pair<T, T>(-1, -1);
  else {
    T lcm = m * n / gcd;
    T x = MOD(a + MOD(n * MOD(_n * ((b - a) / gcd), m / gcd), lcm), lcm);
    return pair<T, T>(x, lcm);
  }
}
```

6.3 Linear Diophantine

```
//FOR SOLVING MINIMUM ABS(X) + ABS(Y)
ll x, y, newX, newY, target = 0;
ll extGcd(ll a, ll b) {
  if(b == 0) {
    x = 1, y = 0;
    return a;
  }
  ll ret = extGcd(b, a % b);
  newX = y;
  newY = x - y * (a / b);
  x = newX;
  y = newY;
  return ret;
}
ll fix(ll sol, ll rt) {
  ll ret = 0;
  //CASE SOLUTION(X/Y) < TARGET
  if(sol < target)
    ret = -floor(abs(sol + target) / (double)rt);
  //CASE SOLUTION(X/Y) > TARGET
  if(sol > target)
    ret = ceil(abs(sol - target) / (double)rt);
  return ret;
}
ll work(ll a, ll b, ll c) {
  ll gcd = extGcd(a, b);
  ll solX = x * (c / gcd);
  ll solY = y * (c / gcd);
  a /= gcd;
  b /= gcd;
  ll fi = abs(fix(solX, b));
  ll se = abs(fix(solY, a));
  ll lo = min(fi, se);
  ll hi = max(fi, se);
  ll ans = abs(solX) + abs(solY);
  for(ll i = lo; i <= hi; i++) {
    ans = min(ans, abs(solX + i * b) + abs(solY - i * a));
    ans = min(ans, abs(solX - i * b) + abs(solY + i * a));
  }
  return ans;
}
```

6.4 Modular Linear Equation

```
// finds all solutions to ax = b (mod n)
vi modular_linear_equation_solver(int a, int b, int n) {
  int x, y;
  vi ret;
  int g = extended_euclid(a, n, x, y);
  if(!(b % g)) {
    x = mod(x * (b / g), n);
    for(int i = 0; i < g; i++)
      ret.push_back(mod(x + i * (n / g), n));
  }
  return ret;
}
```

6.5 Miller-Rabin and Pollard's Rho

```
namespace MillerRabin {
  const vector<ll> primes = { // deterministic up to 2^64 - 1
    2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37
  };
  ll gcd(ll a, ll b) {
    return b ? gcd(b, a % b) : a;
  }
  ll powa(ll x, ll y, ll p) { // (x ^ y) % p
    if(!y)
      return 1;
    if(y & 1)
      return ((__int128) x * powa(x, y - 1, p)) % p;
    ll temp = powa(x, y >> 1, p);
    return ((__int128) temp * temp) % p;
  }
  bool miller_rabin(ll n, ll a, ll d, int s) {
    ll x = powa(a, d, n);
    if(x == 1 || x == n - 1)
      return 0;
    for(int i = 0; i < s; ++i) {
      x = ((__int128) x * x) % n;
      if(x == n - 1)
        return 0;
    }
    return 1;
  }
  bool is_prime(ll x) { // use this
    if(x < 2)
      return 0;
    int r = 0;
    ll d = x - 1;
    while((d & 1) == 0) {
      d >>= 1;
      ++r;
    }
    for(auto& i : primes) {
      if(x == i)
        return 1;
      if(miller_rabin(x, i, d, r))
        return 0;
    }
    return 1;
  }
}

namespace PollardRho {
  mt19937_64 generator(chrono::steady_clock::now()
                       .time_since_epoch().count());
  uniform_int_distribution<ll> rand_ll(0, LLONG_MAX);
  ll f(ll x, ll b, ll n) { // (x^2 + b) % n
    return (((__int128) x * x) % n + b) % n;
  }
  ll rho(ll n) {
    if(n % 2 == 0)
      return 2;
    ll b = rand_ll(generator);
    ll x = rand_ll(generator);
    ll y = x;
    while(1) {
      x = f(x, b, n);
```

```
      y = f(f(y, b, n), b, n);
      ll d = MillerRabin::gcd(abs(x - y), n);
      if(d != 1)
        return d;
    }
  }
  void pollard_rho(ll n, vector<ll>& res) {
    if(n == 1)
      return;
    if(MillerRabin::is_prime(n)) {
      res.push_back(n);
      return;
    }
    ll d = rho(n);
    pollard_rho(d, res);
    pollard_rho(n / d, res);
  }
  vector<ll> factorize(ll n, bool sorted = 1) { // use this
    vector<ll> res;
    pollard_rho(n, res);
    if(sorted)
      sort(res.begin(), res.end());
    return res;
  }
}
```

6.6 Derangement

```
der[0] = 1;
der[1] = 0;
for(int i = 2; i <= 10; ++i)
  der[i] = (ll)(i - 1) * (der[i - 1] + der[i - 2]);
```

6.7 Bernoulli Number

$\sum_{k=1}^{n} k^m = \frac{1}{m+1}\sum_{i=0}^{m}\binom{m+1}{i}B_i^+ n^{m+1-i} = m!\sum_{i=0}^{m}\frac{B_i^+ n^{m+1-i}}{i!(m+1-i)!}$

$B_n^+ = 1 - \sum_{i=0}^{n-1}\binom{n}{i}\frac{B_i^+}{n-i+1}, \quad B_0^+ = 1$

6.8 Forbenius Number

(X* Y) - (X + Y) and total count is(X - 1)* (Y - 1) / 2

6.9 Stars and Bars with Upper Bound

$P = (1 - X^{r_1+1})\dots(1 - X^{r_n+1}) = \sum_i c_i X^{e_i}$

$Ans = \sum_i c_i \binom{N-e_i+n-1}{n-1}$

6.10 Arithmetic Sequences

$U_n = a + (n-1)a_1 + \frac{(n-1)(n-2)}{1\times2}a_2 + \dots + \frac{(n-1)(n-2)(n-3)\dots}{1\times2\times3\times\dots}a_r$

$S_n = n \times a + \frac{n(n-1)}{1\times2}a_1 + \frac{n(n-1)(n-2)}{1\times2\times3}a_2 + \dots + \frac{n(n-1)(n-2)(n-3)\dots}{1\times2\times3\dots}a_r$

6.11 Basis Vector

```
int basis[d]; // basis[i] keeps the mask of the vector whose f value is i
int sz;       // Current size of the basis
void insertVector(int mask) {
  for(int i = 0; i < d; i++) {
    if((mask & 1 << i) == 0) {
      continue; // continue if i != f(mask)
    }
    if(!basis[i]) {  // If there is no basis vector with the i'th bit set,
      // then insert this vector into the basis
      basis[i] = mask;
      ++sz;
      return;
    }
    mask ^= basis[i]; // Otherwise subtract the basis vector from this
    // vector
  }
}
```

7 Strings

7.1 KMP

```
auto get_kmp = [&](string S, string T) -> vector<int> {
  // S is the text, T is the pattern // ababa aba -> expected: {1 2 3 2 3}
  int N = S.size(), M = T.size();
  vector<int> lps(M), kmp(N);
  for(int i = 1, j = 0; i < M;) {
    if(T[i] == T[j])
      lps[i++] = ++j;
    else {
      if(j)
        j = lps[j - 1];
      else
        lps[i++] = 0;
    }
  }
  for(int i = 0, j = 0; i < N;) {
    if(S[i] == T[j]) {
      kmp[i++] = ++j;
      if(j == M)
        j = lps[j - 1];
    } else {
      if(j)
        j = lps[j - 1];
      else
        kmp[i++] = 0;
    }
  }
  return kmp;
};
```

7.2 Z Function

```
auto z_function(string s) {
    int n = s.size();
    vector<int> z(n);
    int l = 0, r = 0;
    for(int i = 1; i < n; i++) {
        if(i < r) {
            z[i] = min(r - i, z[i - l]);
        }
        while(i + z[i] < n && s[z[i]] == s[i + z[i]]) {
            z[i]++;
        }
        if(i + z[i] > r) {
            l = i;
            r = i + z[i];
        }
    }
    return z;
}
```

8 OEIS

8.1 A000108 (Catalan)

```
Catalan numbers
f(n) = nCk(2n,n) / (n+1) = nCk(2n,n) - nCk(2n,n+1) = f(n-1) * 2*(2*n-1) / (n+1)
1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, 208012, 742900,
2674440, 9694845, 35357670, 129644790, 477638700, 1767263190, 6564120420,
24466267020, 91482563640, 343059613650, 1289904147324, 4861946401452,
18367353072152, 69533550916004, 263747951750360, 1002242216651368,
3814986502092304
```

8.2 A018819

```
Binary partition function: number of partitions of n into powers of 2
f(2m+1) = f(2m);  f(2m) = f(2m-1) + f(m)
1, 1, 2, 2, 4, 4, 6, 6, 10, 10, 14, 14, 20, 20, 26, 26, 36, 36, 46, 46, 60,
60, 74, 74, 94, 94, 114, 114, 140, 140, 166, 166, 202, 202, 238, 238, 284,
284, 330, 330, 390, 390, 450, 450, 524, 524, 598, 598, 692, 692, 786, 786,
900, 900, 1014, 1014, 1154, 1154, 1294, 1294
```

8.3 A092098

```
3-Portolan numbers: number of regions formed by n-secting the angles of
an equilateral triangle.
long long solve(long long n) {
    long long res = (n % 2 == 1 ? 3*n*n - 3*n + 1 : 3*n*n - 6*n + 6);
    const int bats = n/2 - 1;
    for (long long i=1; i<=bats; i++) for (long long j=1; j<=bats; j++) {
        long long num = i * (n-j) * n;
        long long denum = (n-i) * j + i * (n-j);
        res -= 6 * (num % denum == 0 && num / denum <= bats);
    } return res;
}
1, 6, 19, 30, 61, 78, 127, 150, 217, 246, 331, 366, 469, 510, 625, 678, 817,
870, 1027, 1080, 1261, 1326, 1519, 1566, 1801, 1878, 2107, 2190, 2437, 2520,
2791, 2886, 3169, 3270, 3559, 3678, 3997, 4110, 4447, 4548, 4921, 5034, 5419,
5550, 5899, 6078, 6487
```

8.4 A000127

```
Maximal number of regions obtained by joining n points around a circle by
straight lines
f(n) = (n^4 - 6*n^3 + 23*n^2 - 18*n + 24) / 24
1, 2, 4, 8, 16, 31, 57, 99, 163, 256, 386, 562, 794, 1093, 1471, 1941, 2517,
3214, 4048, 5036, 6196, 7547, 9109, 10903, 12951, 15276, 17902, 20854, 24158,
27841, 31931, 36457, 41449, 46938, 52956, 59536, 66712, 74519, 82993, 92171,
102091, 112792, 124314
```

8.5 A001534

```
Number of graphs with n nodes and n edges.
0, 0, 1, 2, 6, 21, 65, 221, 771, 2769, 10250, 39243, 154658, 628635, 2632420,
11353457, 50411413, 230341716, 1082481189, 5228952960, 25945377057,
132140242356, 690238318754
```